Chapter 2 - Part 3
Handling missing values

Data can be missing bcuz of human errors

Instructor: Dr. Heyfa Ammar

Slides courtesy: Dr. Anis Koubaa

Handling Missing Data in Python

Why Handle Missing Data?

- Missing data can distort analysis and lead to biased results.
- Proper handling ensures data integrity and better model performance.

	student_id	name	major01	major02	project	final
0	120060258	Mohamed Salah	0.25	0.00	0.000	13.0
1	211110888	NaN	15.22	10.75	14.670	27.0
2	211210095	Ahmed Assiri	13.25	NaN	16.250	NaN
3	212110756	Abdullah Ahmari	5.91	1.25	5.000	7.0
4	212110889	Faisal Ali	16.00	NaN	6.670	NaN
5	212110977	NaN	10.31	12.45	15.758	33.0
6	213110290	Abdulaziz Samir	15.00	NaN	NaN	NaN
7	213110419	Anas Karim	16.00	12.63	16.670	23.0
8	213110460	Mahmoud Adel	10.00	6.38	NaN	8.0
9	213110964	NaN	10.00	7.50	11.670	15.0
10	213110964	Hany Abdullah	9.91	NaN	5.330	30.0
11	213111074	Abdelhalim Mohanned	10.56	12.00	11.330	20.0
12	213111128	NaN	19.00	17.25	18.000	NaN
13	213210082	Khaled Majid	20.00	18.88	18.670	37.0
14	213254033	Saad Ali	10.00	NaN	NaN	3.0

Key Alternatives for Handling Missing Data

1. **Drop Missing Data**
   - **When**: If missing values are few or non-critical.
   - **Impact**: Reduces dataset size but avoids introducing bias.
2. **Fill Missing Data (Imputation)**
   - **Options**: Replace missing values with the mean, median, or a constant.
   - **Impact**: Retains all data, but assumptions must be valid to avoid bias.
3. **Forward/Backward Fill**
   - **When**: Use prior or subsequent data points to fill gaps.
   - **Impact**: Best for time series or sequential data.

Identifying Missing Values

- **Checking for Missing Data**:
  - Use `.isnull()` to identify missing values in the DataFrame:

```python
df.isnull()
```

Output (partial):

student_id	name	major01	major02	project	final
120060258	False	False	False	False	False
211110888	True	False	False	False	False
211210095	False	False	True	False	True

Summary of Missing Values

- Use `.isnull().sum()` to get the total count of missing values per column:

```python
df.isnull().sum()
```

Copy code

Output:

Column	Missing Values
name	5
major01	0
major02	5
project	3
final	4

Reading the damage report:

- `major01` has **zero** missing values — every student has a first-major score.
- `name`, `major02`, `project`, and `final` all have gaps, ranging from a handful of rows to nearly a third of the class.

Before deciding what to do, it's worth asking *why* a value might be missing:

Pattern	Meaning	Example here
MCAR — Missing Completely at Random	The gap has nothing to do with any value, observed or not	A sync glitch dropped a random cell
MAR — Missing at Random	The chance of missing depends on *other observed* data	Late-registering students are missing `major01` because they hadn't taken it yet
MNAR — Missing Not at Random	The chance of missing depends on the *missing value itself*	A student who failed the `final` skipped submitting it out of embarrassment

No correlation

There is a reason

You can't always tell which case you're in from the data alone — but the distinction shapes whether dropping or filling is defensible. Blindly filling MNAR data with the mean can hide exactly the students who most need attention.

Understanding the missing values is important to be able to choose the strategy needed to fix

Step 3 — Pick a strategy, column by column

There is no single "correct" way to handle missing data — the right move depends on **what the column means** and **how much is missing**. Here's the call you're going to make for this gradebook:

Column	% missing	Strategy	Why
`name`	small	**Drop those rows**	You cannot report a grade for a student you cannot identify — no value can substitute for an identity
`major01`	0%	**Nothing to do**	Complete already — No need to fix it
`major02`	moderate	**Fill with the column mean**	A continuous score close to normally distributed; mean imputation preserves the class average for downstream stats — The mean will be used
`project`	moderate	**Fill with the column median**	Project scores are more skewed (a few 0s from late submissions) — median resists those outliers better than the mean — Median will be used
`final`	small–moderate	**Drop those rows**	The final exam is the single highest-stakes score — guessing it for a registrar submission is not defensible — Can't use anything to guess the number

This mirrors the three big levers you have for any dataset:

1. **Drop** — safest when missingness is small and the value can't be reasonably guessed
2. **Fill with a constant / statistic** — good for continuous data where a placeholder is defensible and you need to keep every row
3. **Forward/backward fill** — best for ordered or time-series data, which we'll demo separately below

Let's apply each technique — starting with the building blocks, then the final pipeline.

Strategy A — Dropping missing values with `dropna()`

Filling Missing Values (fillna())

- **Filling with a Constant Value:**
  - Replace all missing values in the `final` column with `0`:

```python
df['final'].fillna(0, inplace=True)
```

Modify in the data frame itself but u can copy it and modify the copied on

Example Before:

name	final
Ahmed Assiri	NaN
Khaled Majid	37.0

Example After:

name	final
Ahmed Assiri	0.0
Khaled Majid	37.0

Filling Missing Values (fillna())

Filling with the Mean or Median:

- Fill missing values in `major02` with the column's **mean**:

```python
df['major02'].fillna(df['major02'].mean(), inplace=True)
```

Copy code

Example Before:

name	major02
Ahmed Assiri	NaN
Mahmoud Adel	6.38

Example After:

name	major02
Ahmed Assiri	11.73
Mahmoud Adel	6.38

Dropping Missing Values (dropna())

- **Drop Rows with Missing Values**:
  - Remove rows where **any** column has a missing value:

```python
df.dropna(inplace=True)
```

Example Before:

name	final
Faisal Ali	NaN
Hany Abdullah	30.0

Example After (Faisal Ali's row is dropped):

name	final
Hany Abdullah	30.0

- **Drop Rows with Missing Values in Specific Columns**:
  - Drop rows where the `final` score is missing:

```python
df.dropna(subset=['final'], inplace=True)
```

Dropping Missing Values (dropna())

- **Remove Rows/Columns with Missing Data:**
  - Use `dropna()` to remove rows or columns with missing values.

```python
df.dropna(axis=0, inplace=True)  # Drops rows
df.dropna(axis=1, inplace=True)  # Drops columns
```

  - Customize with `thresh=` to keep rows/columns with a minimum number of non-null values.

Dropping Missing Values (dropna())

```python
import pandas as pd

# Sample DataFrame
df = pd.DataFrame({
    'A': [1, 2, None, 4],
    'B': [None, 2, 3, 4],
    'C': [1, None, 3, 4]
})

print('df\n',df,'\n')

# Detect missing values
missing_values = df.isnull()
# Removing rows with missing data
df_dropna = df.dropna()
print('df_dropna\n', df_dropna,'\n')

# Filling missing data with a value
df_filled = df.fillna(value=0)
print('df_filled\n',df_filled,'\n')

# Filling with the column mean
df_fillmean = df.fillna(df.mean())
print('df_fillmean\n',df_fillmean,'\n')

```

```
df
     A    B    C
0  1.0  NaN  1.0
1  2.0  2.0  NaN
2  NaN  3.0  3.0
3  4.0  4.0  4.0

df_dropna
     A    B    C
3  4.0  4.0  4.0

df_filled
     A    B    C
0  1.0  0.0  1.0
1  2.0  2.0  0.0
2  0.0  3.0  3.0
3  4.0  4.0  4.0

df_fillmean
          A    B         C
0  1.000000  3.0  1.000000
1  2.000000  2.0  2.666667
2  2.333333  3.0  3.000000
3  4.000000  4.0  4.000000
```